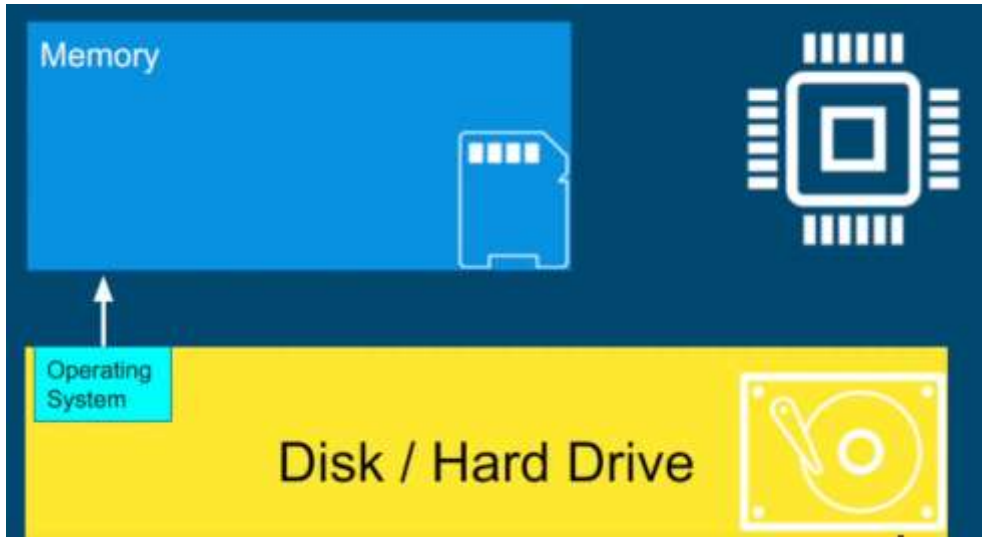
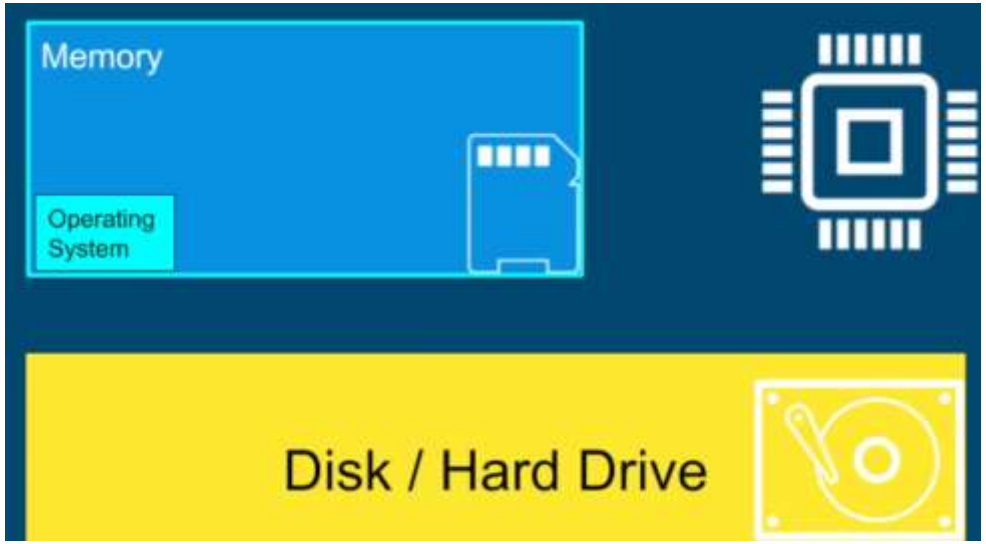
Linux Programming

Atul Kahate

Why Operating Systems?

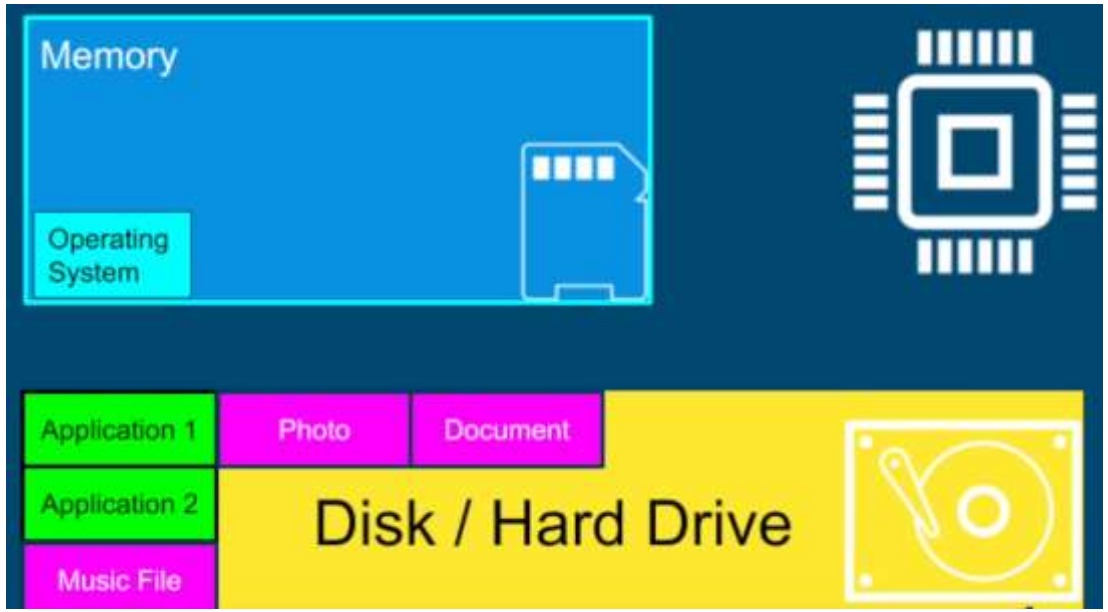
- When we start our computer, a special program called **operating system** is loaded from the disk into the main memory (RAM) of the computer

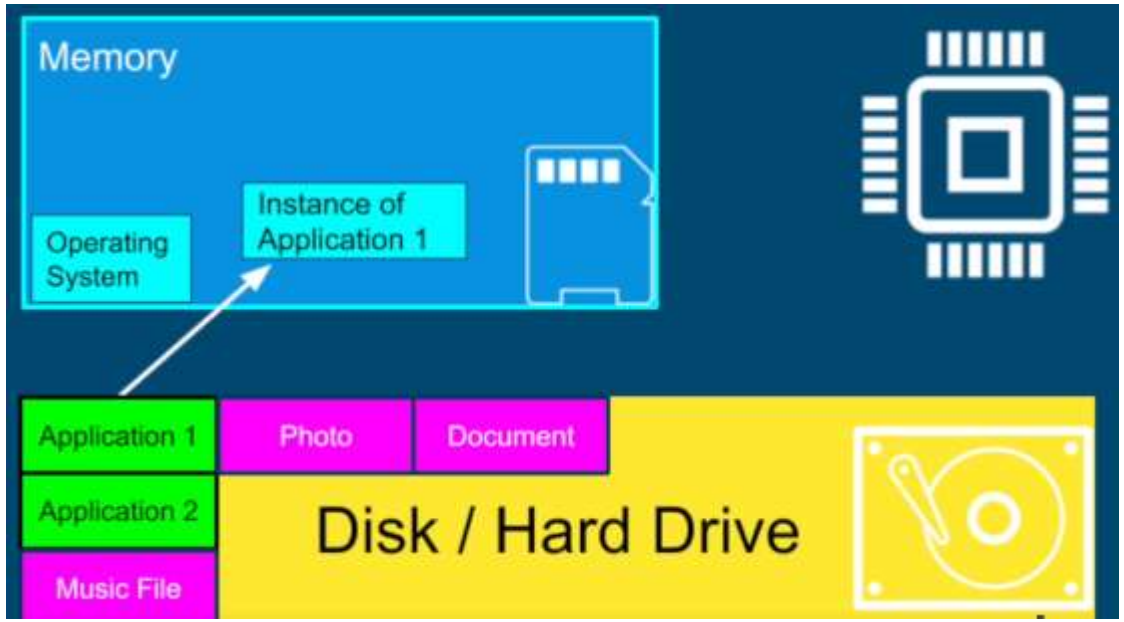




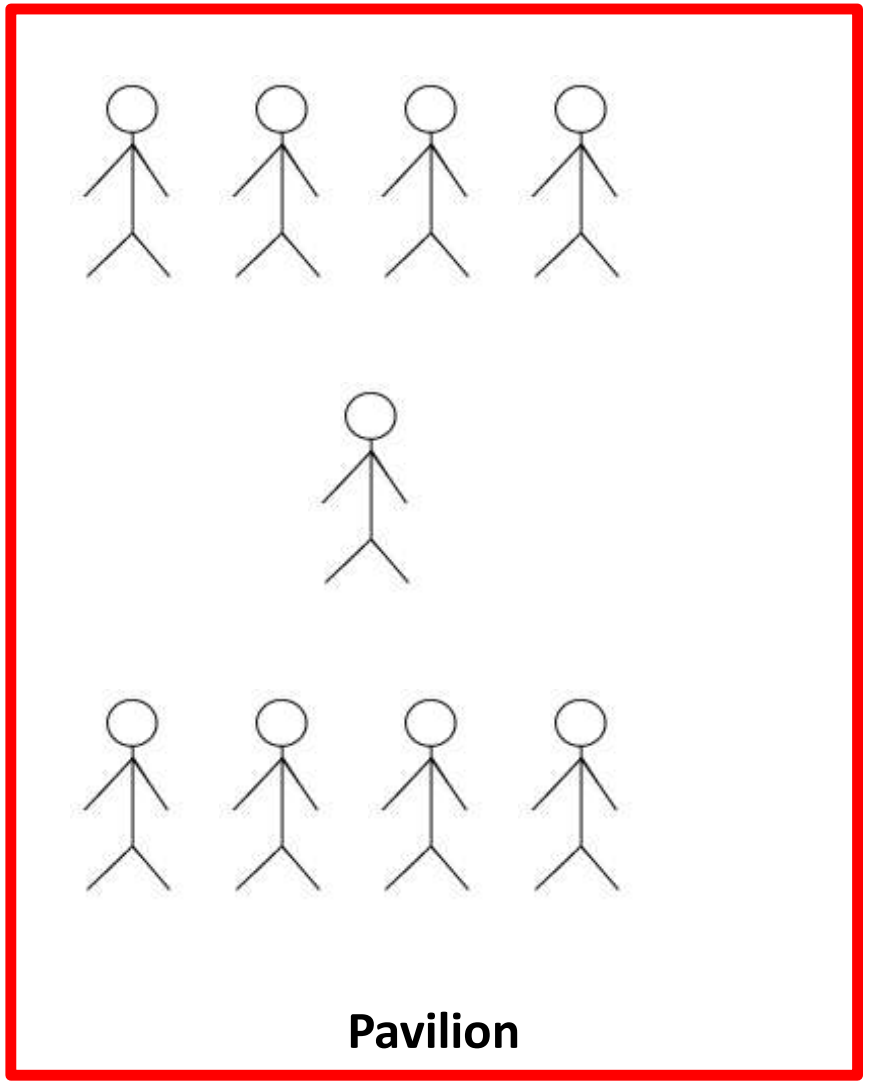
Why Operating Systems?

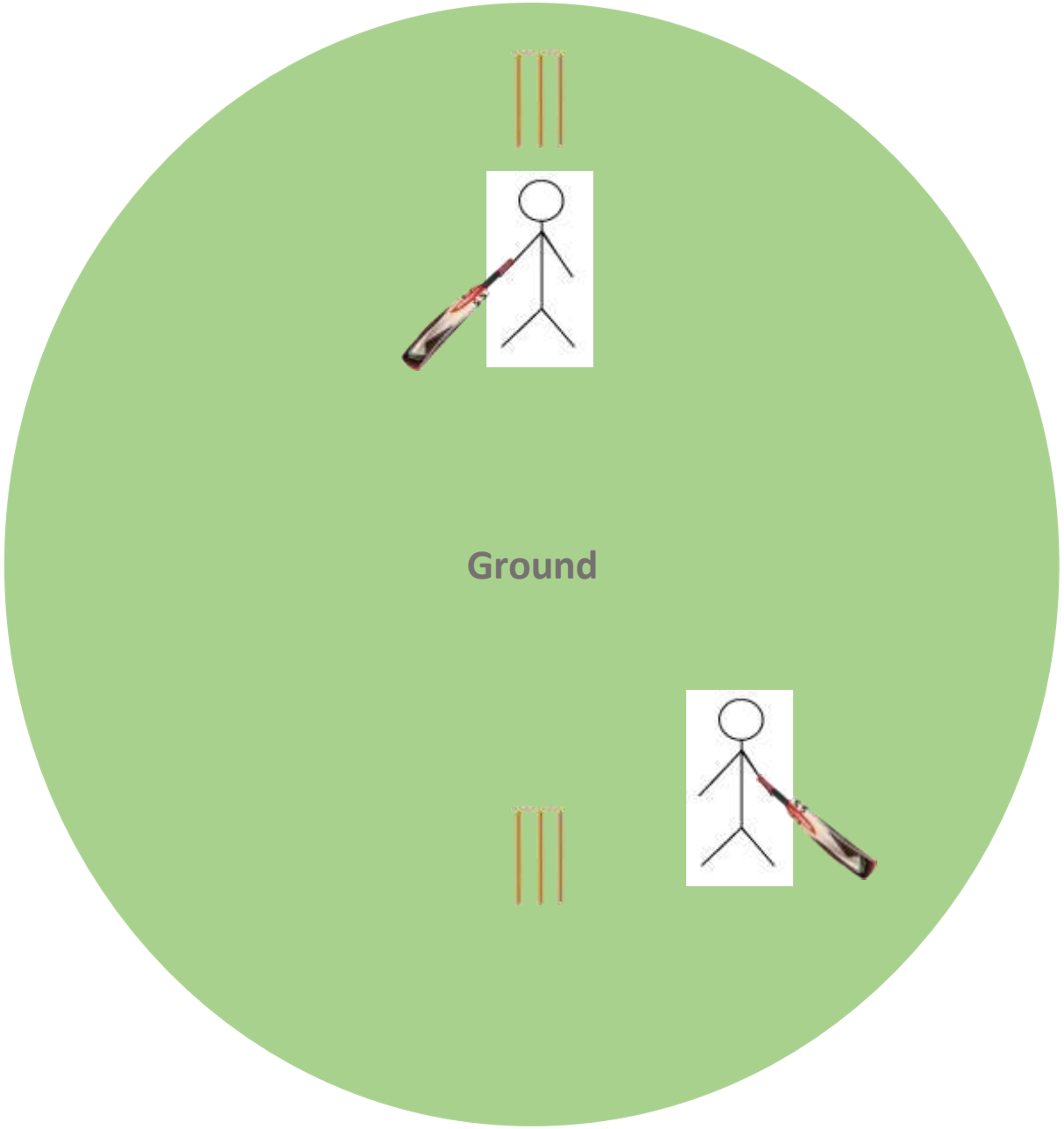
- All our applications are on the disk in the form of files
- When we run an application, it is loaded by the operating system into the main memory





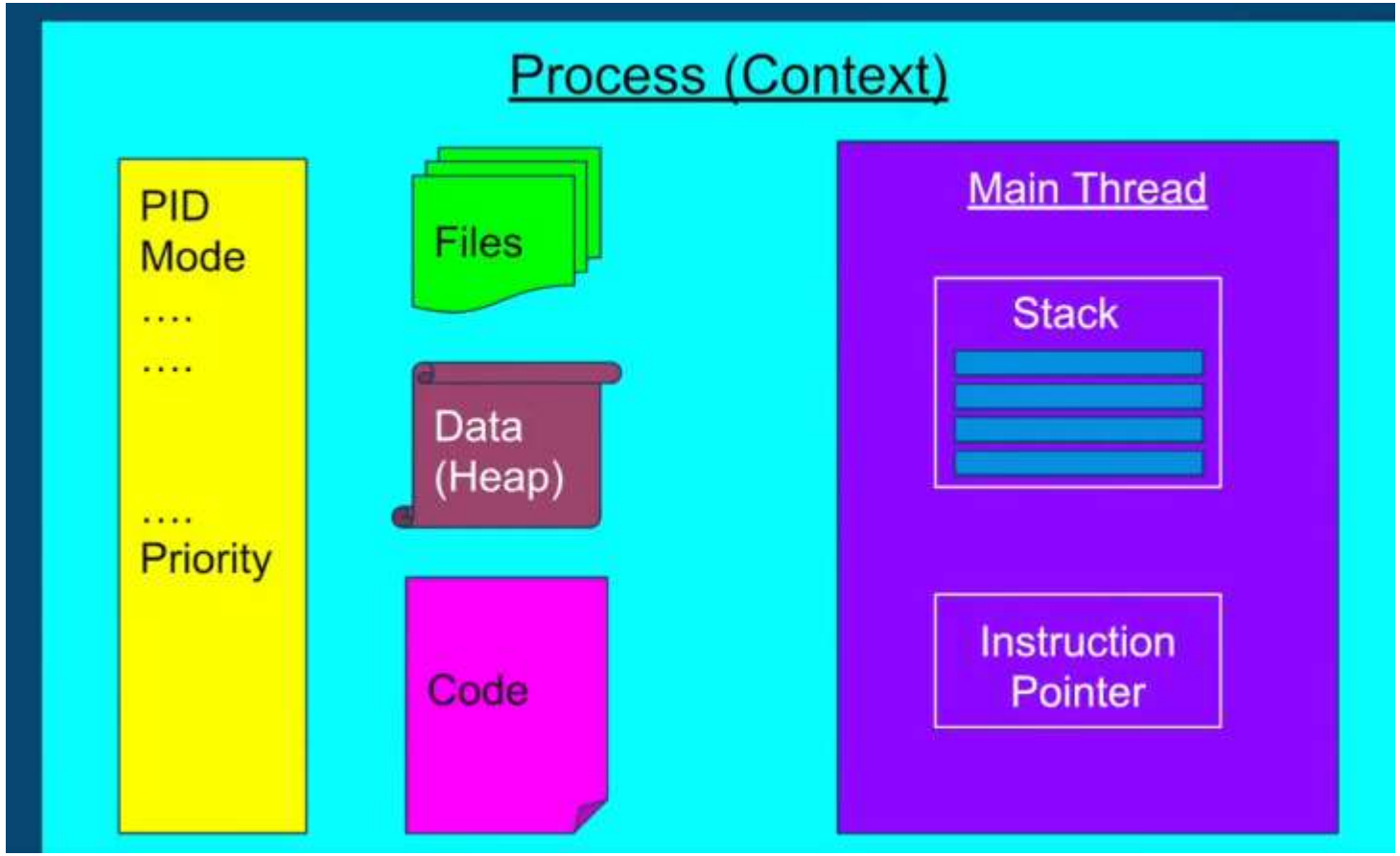
Program and Process





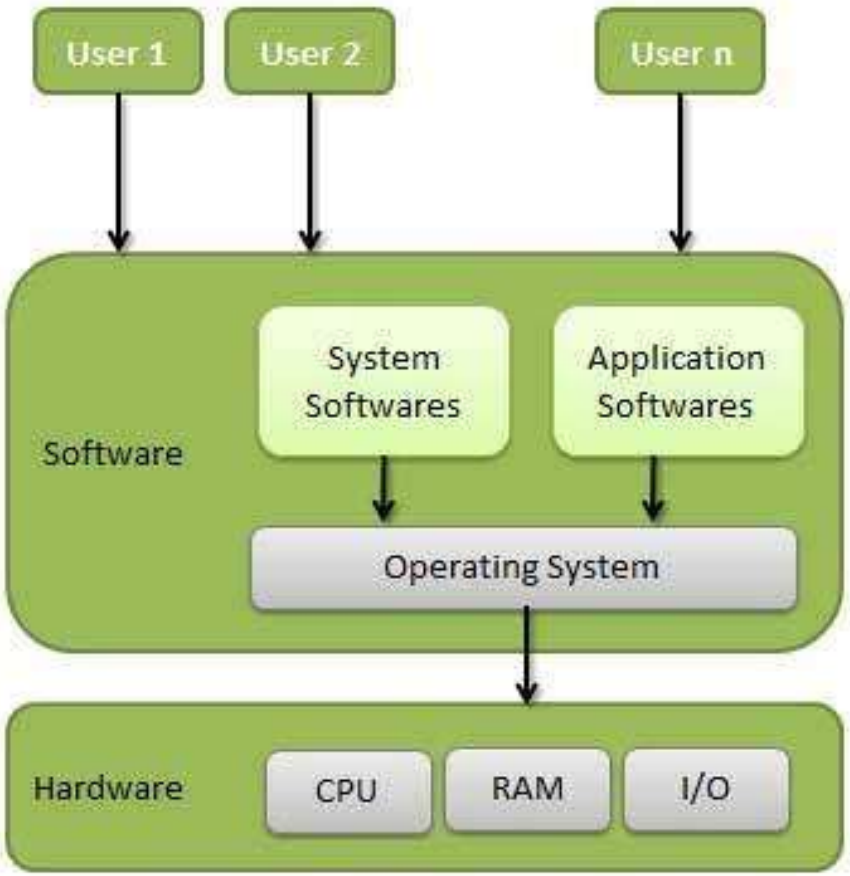
Why Operating Systems?

- The instance of a program is a **process**



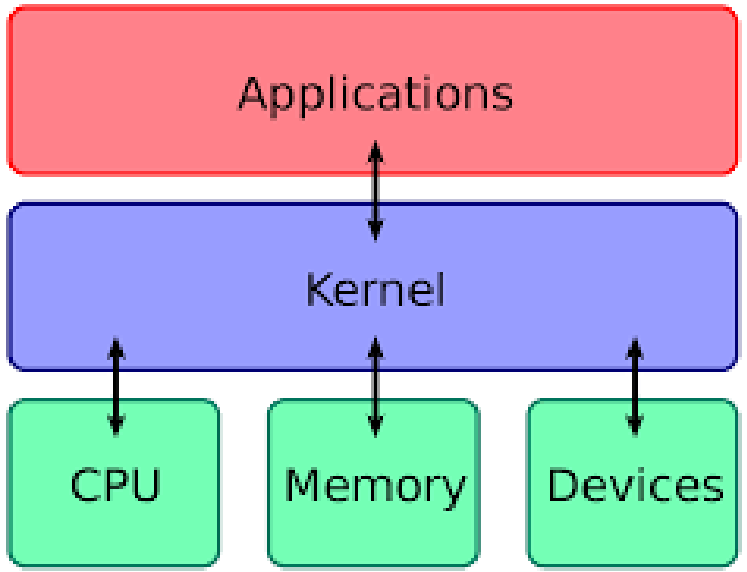
Introduction to OS Concepts

- **Operating System (OS)**: A program that acts as an bridge between a user and the computer hardware
- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner



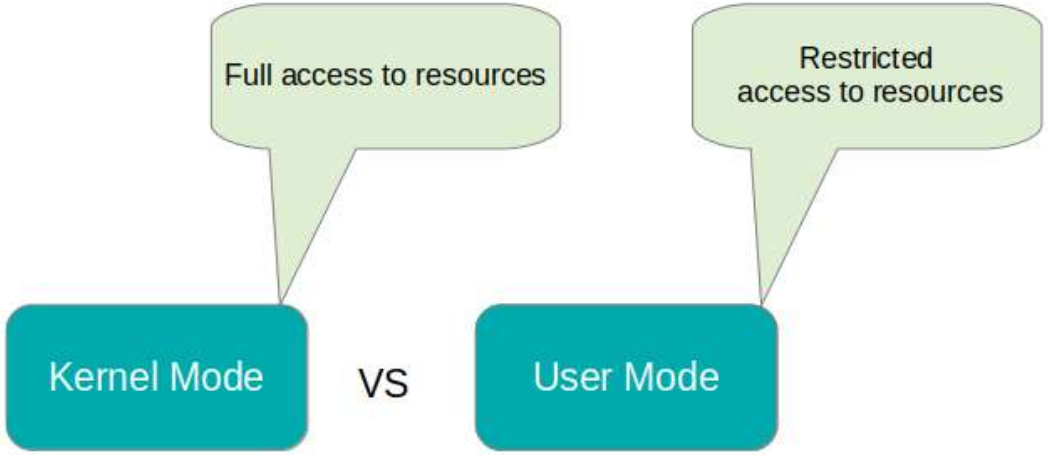
Kernel

- “The one program running at all times on the computer” is the **kernel**
- Everything else is either a **system program** (ships with the operating system) or an **application program**



User Mode and Kernel Mode

- **User mode**
  - Restricted access to resources
  - If a resource such as disk or file is needed, a signal is sent to the CPU via an interrupt to switch to the kernel mode
- **Kernel mode** (also called as **supervisor mode**)
  - Full access to memory, I/O, and other resources



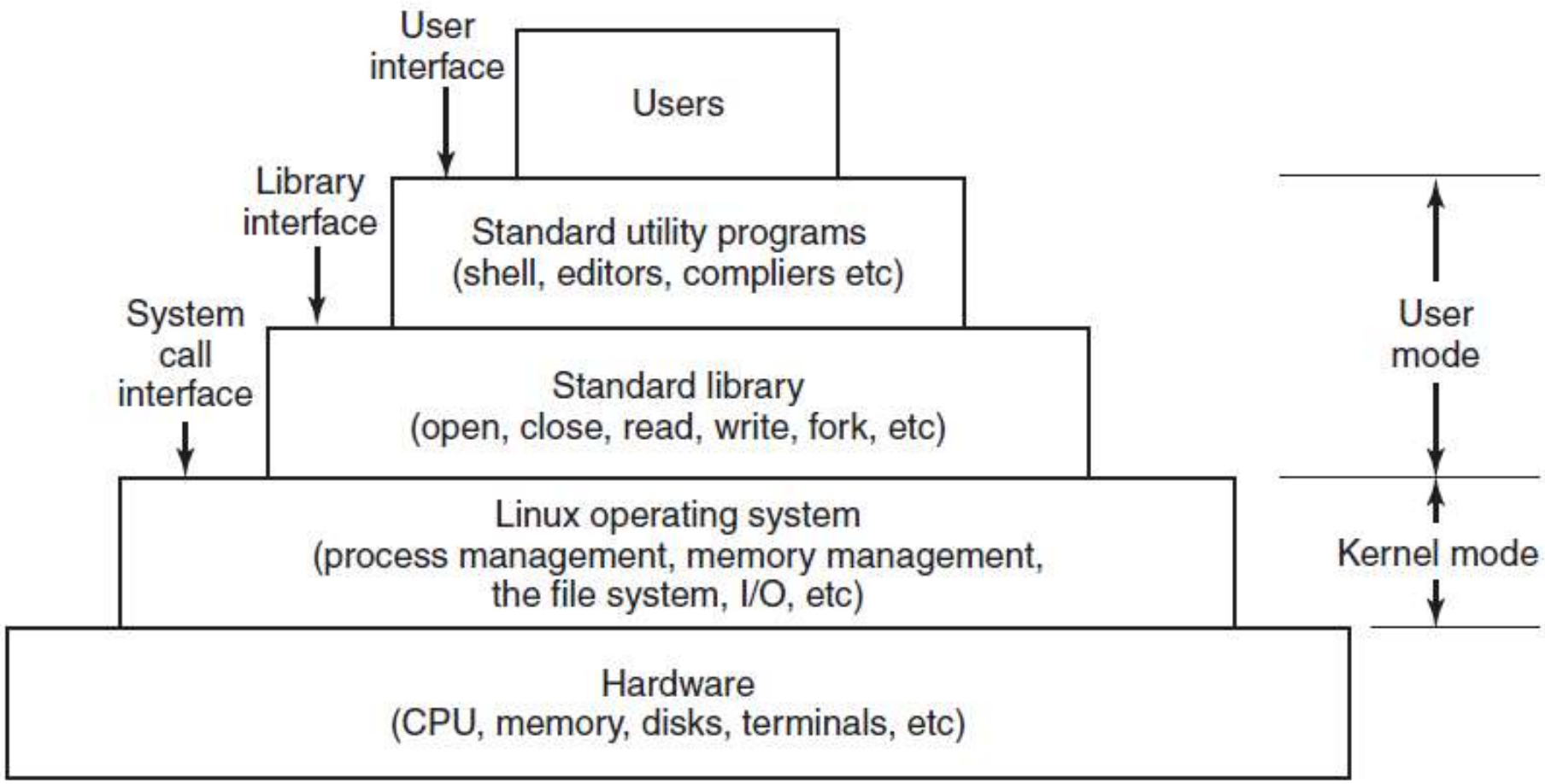
Linux History

- **Linux**: World’s most widely used operating system
- Open source
- Created by Linus Torvalds in 1991
- Origins in UNIX (1960s)

GNU, GPL

- **GNU**: **G**NU is **N**ot **U**nix – Announced by Richard Stallman in 1983
- **Free Software Foundation (FSF)** – Founded by Stallman
- “Free” software – Free to use, study, modify, and redistribute
- Collection of about 385 packages as of September 2023
- **GPL**: General Public License, also called **Copyleft**
- Linus Torvalds released the Linux kernel in 1991 under GNU GPL

Layers in Linux



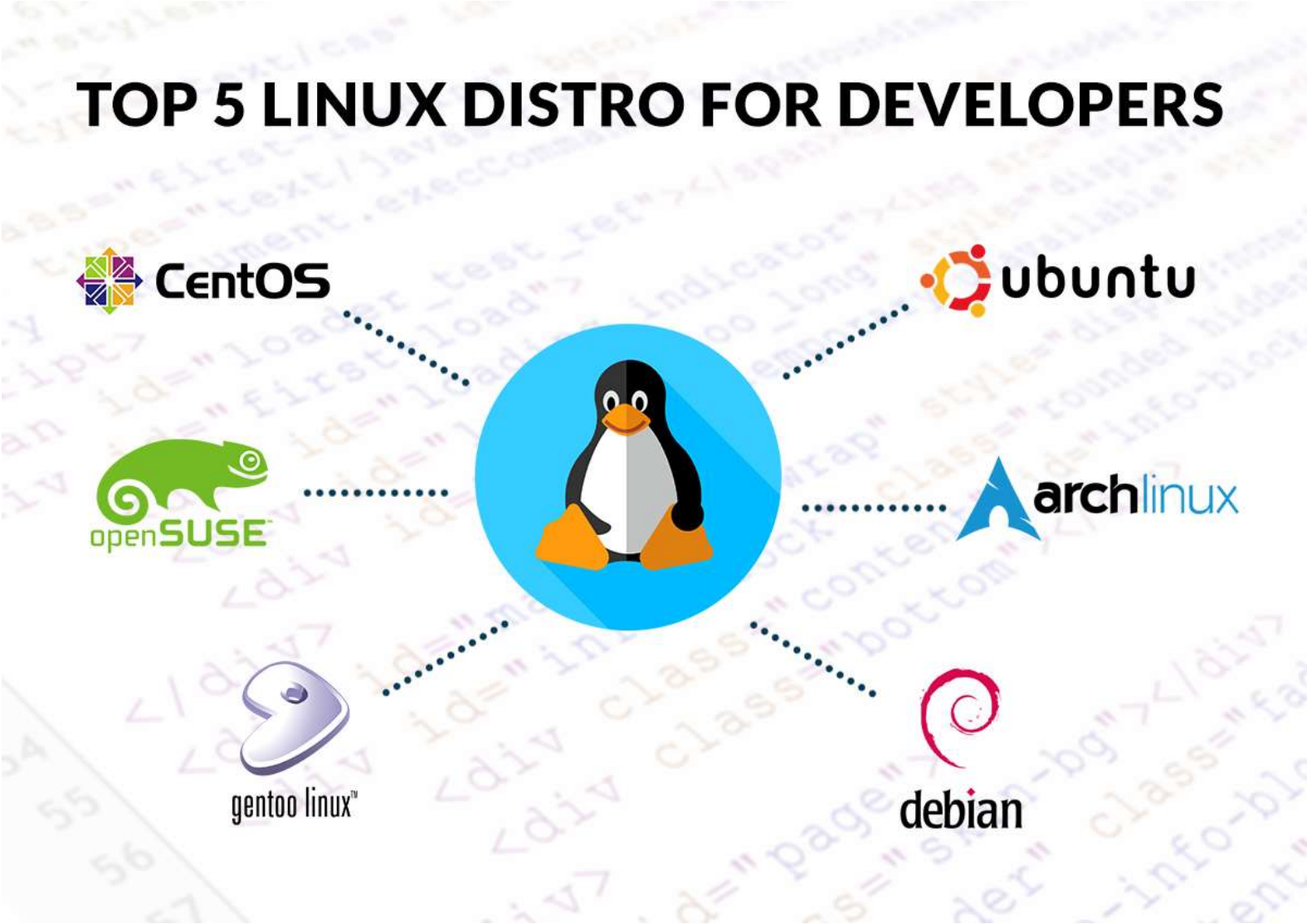
Installing Linux – Multiple Options

- On a Windows machine, install WSL (Simplest and light-weight)
  - https://learn.microsoft.com/en-us/windows/wsl/install

- Use a virtual machine and install a Linux distribution
  - VMWare or Oracle VirtualBox (See https://techbland.com/how-to-install-lubuntu-on-virtualbox-on-windows/)

- Create a dual-boot machine (Windows and Linux)

Linux Distributions (“Distros”)



Installing Ubuntu/CentOS

- Download ISO file (Disk image file format): https://lubuntu.me/downloads/ or https://www.centos.org/download/
- Install this in VirtualBox/VMWare

Run Levels

- Run level = Operating mode (Examples: Single-user, Multi-user, Networking, Shutdown)
- Not used in the modern systems

Linux Command Line

- Command line = **Shell**

Shell	Program Name	Description
Bourne shell	sh	Original shell program from AT&T, available on all UNIX systems
C shell	csh	Shell developed as a part of BSD UNIX
TC shell	tsh	Similar to csh, with some enhancements
Korn shell	ksh	AT&T's improvement of the Bourne shell
Bourne Again Shell	**bash**	**Shell distributed with Linux**

The Shell Prompt

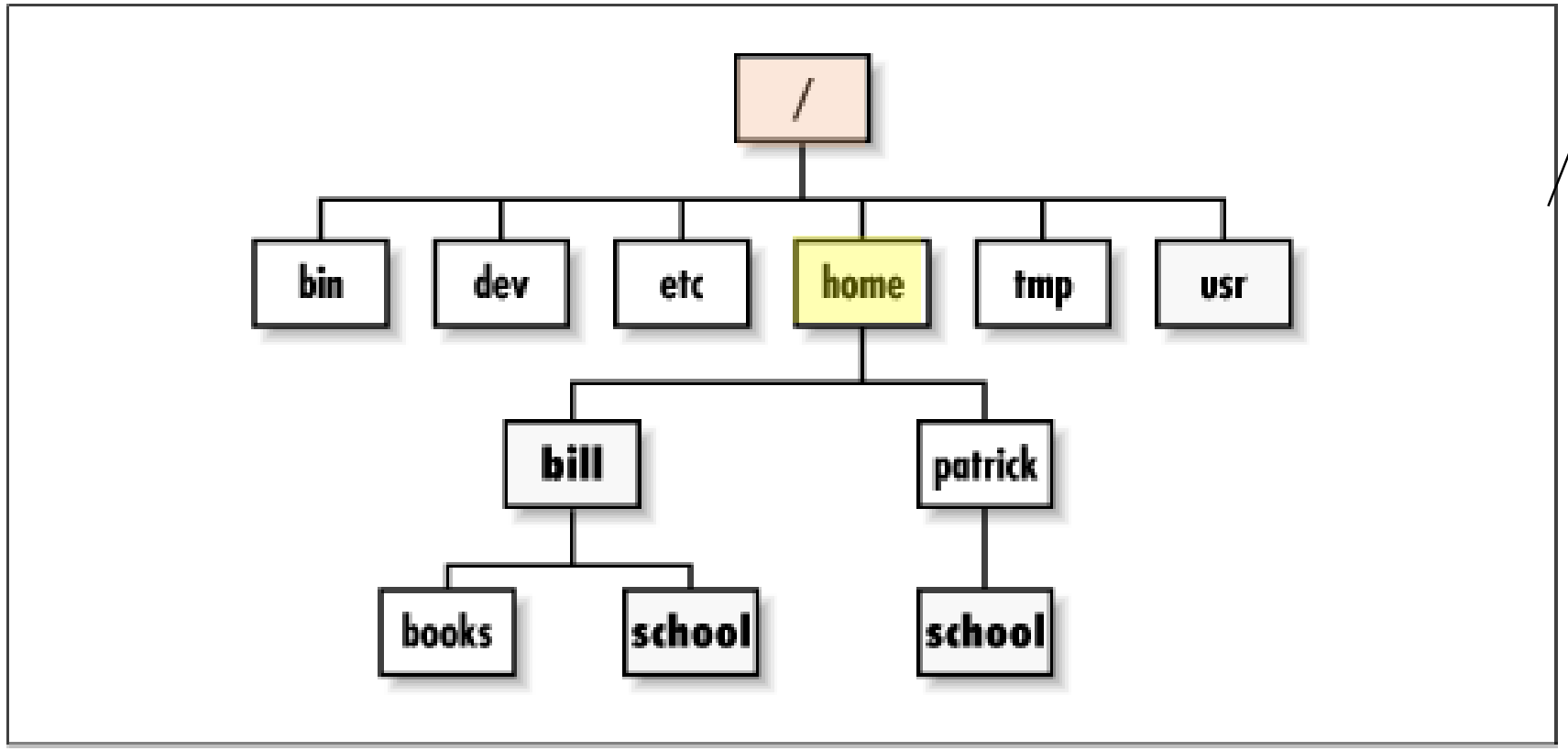
```
atul@lubuntu:~$
```

Component	Example	Description
Username	atul	The account you are currently logged into.
Separator	@	Separates the user from the machine name.
Machine Name	lubuntu	The name of the computer or server (hostname).
Directory	~	Your current location. ~ represents your Home folder.
Indicator	$ or #	Shows your privilege level (User vs. Root).

File and Directory Management

Directory Structure

- / is the **root directory**
- Each user on the system has a sub-directory inside *home*



symbol (~) represents the user's home

Navigation and Path-Related Commands

Command	Description	Example
pwd	Print Working Directory (Shows your current location)	pwd
ls	List files and directories	ls -lh (human-readable sizes)
cd	Change directory	cd /var/www
cd ..	Move up one level to the parent directory	cd ..
cd ~	Go directly to the current user's home directory	cd ~
cd -	Switch back to the previous directory you were in	cd -

File and Directory Commands

Command	Description	Example
touch	Create a new empty file	touch notes.txt
mkdir	Create a new directory (folder)	mkdir project_files
cp	Copy files or directories	cp file.txt backup.txt
cp -r	Copy a directory and all its contents (recursive)	cp -r folder/ backup_folder/
mv	Move or Rename a file/directory	mv old_name.txt new_name.txt
rm	Remove (delete) a file	rm temp.log
rm -r	Remove a directory and its contents (be careful!)	rm -r old_data/

Viewing and Searching Files

Command	Description	Example
cat	Display the entire content of a file	cat config.conf
less	Open a file for interactive reading (allows scrolling)	less large_log.log
head	Show the first 10 lines of a file	head -n 20 file.txt
tail	Show the last 10 lines of a file	tail -f system.log (live view)
grep	Search for a specific pattern/text inside a file	grep "Error" server.log
find	Search for files and folders based on name or type	find . -name "*.pdf"

System Information and Shortcuts

Shortcut/Cmd	Description
Tab	**Auto-complete**: Type half a name and hit Tab to finish it.
history	See a list of all commands you have run recently.
clear	Clears the terminal screen (or use Ctrl + L).
man	The "Manual"—use it to see instructions for any command (e.g., man ls).
Ctrl + C	Stop the currently running command/process immediately.

File and Directory Commands

- **touch**: Create an empty file
- **pwd**: Print Working Directory
- **mkdir**: Create a directory
- **cd**: Change directory
  - Absolute path: Start from root – cd /home/atul/Downloads
  - Relative path: Start from current position – cd .. or cd test
  - Go to root: cd /
  - Go to user's home directory: cd
- *Note: . is the current directory and .. Is the parent directory*
- **ls**: List all files in a specified directory
- **cp**: Copy files and directories
- **mv**: Rename a file/directory
- **rm**: Delete a file, **rm -r**: Recursively delete a directory and all sub-directories

More Linux Commands

- **man**: Display manual pages for a command, e.g. *man sort*
- **sort**: Sort the lines of a text file or input stream alphabetically or numerically, e.g. *sort names.txt* or *sort -r names.txt*
- **ln**: Create links between files/directories, e.g. *ln -l abc.txt myhardlink* or *ln -s abc.txt mysoftlink*
- **gzip**: Compress single files, e.g. *gzip file1*
- **zip**: Compress files into an archive, e.g. *zip myzipped.zip file1 file2 file3*
- **tar**: Some differences, e.g. *tar -cvf archive.tar file1 file2*
- **ps**: Status of currently running processes
- **kill**: Terminate a process, e.g. *kill 1234* or *pkill firefox*
- **top**: Interactive utility to monitor processes

Built-ins

- **Built-in**s: Commands implemented within the shell itself (e.g. cd, echo, pwd
- How to find if a command is a built-in?
- **type** <command> … e.g. type cd

Assignment

1. Create an empty file cdac.txt. [1 mark]
2. Write the data "Welcome to CDAC." (Use any Linux command but don’t use an editor)
3. Append the data "Welcome to DAC." in cdac.txt.

Assignment

- Problem: As root, create a directory dbda and under it create a directory named test and create 100 files under it with name file1, file2..file100 - all this using a single command.
- Solution: **sudo mkdir -p /dbda/test && sudo sh -c 'cd /dbda/test && touch file{1..100}’**

- sudo mkdir -p /dbda/test: This command creates the directory /dbda/test along with any necessary parent directories (-p option) with root privileges.
- &&: This operator allows us to execute multiple commands sequentially.
- sudo sh -c 'cd /dbda/test && touch file{1..100}': This part of the command changes the directory to /dbda/test and then uses the touch command with brace expansion to create 100 files named file1 through file100 in that directory, all within a single subshell (sh -c). The sudo command is used to execute the subshell with root privileges.

Linux Editors

- vi – The original Linux editor
- vim – Improved version of vi
- nano – Easy to work with (To install: **sudo apt-get update** and then **sudo apt-get install nano**)

The vi Editor

- Modal editing: Three different modes: normal, insert, command-line
- **Normal mode**: Navigate the file, Editing commands are possible
- **Insert mode**: For inserting text
- **Command-line mode**: Editor commands

- Normal mode commands: Cursor movement using h (left), j (down), k (up), l (right), w (one word forward), b (one word back), gg (BOF), G (EOF)

Using vi

- Problem: Create a file test.txt, add three lines, save and exit and again reopen the file and edit the second line, save and exit.
- By default, vi starts in the normal mode, so press **i** to switch to insert mode and type three lines, then **ESC**, then type **:wq** and **ENTER** to save and exit
- Then reopen the file, use **j** twice to move cursor down two lines, press **i** to switch to insert mode, again repeat **ESC :wq ENTER**
- To quit vi without saving the changes, **ESC :q! ENTER**
- To display line numbers in vi, **ESC :set number ENTER**
- Normal mode commands: Cursor movement using h (left), j (down), k (up), l (right), w (one word forward), b (one word back), gg (BOF), G (EOF)

Move First Three Lines of a File to its End in vi

- Once inside vi, press the Esc key to ensure you are in command mode.
- Move the cursor to the beginning of the file by pressing gg.
- Enter visual mode by typing V (uppercase V).
- Move the cursor down to select the first three lines by pressing 3j or simply 3↓.
- Once the lines are selected, cut them by typing d.
- Move to the end of the file by typing G.
- Paste the cut lines by typing p.

Copy the 5th line and paste it above the first line in vi

- Once inside vi, press the Esc key to ensure you are in command mode.
- Move the cursor to the 5th line by typing 5G or simply 5gg.
- Enter visual mode by typing V (uppercase V).
- Move the cursor down to select the entire 5th line by pressing j or ↓.
- Once the line is selected, copy it by typing y.
- Move to the first line by typing gg.
- Paste the copied line above the first line by typing P (uppercase P).

Find and Replace in vi

- Forward search cdac: ESC and then /cdac (n to go to the next occurrence, N to go to the previous occurrence)
- Backward search cdac: ESC and then ?cdac (n to go to the previous occurrence, N to go to the next occurrence)

- Replace abc with xyz: ESC and then **:%s/abc/xyz/g**
- Note: g = global, if omitted, it will replace only the first occurrence

The who Command

- Display all users: **who**

- Possible question: Combine the outputs of the who and pwd commands:
- **who && pwd**
- The && operator **chains** multiple commands, second runs only if the first one succeeds

Examples

Question	Command	Key points
List all the files and sub directories of the directory /bin.	**ls -la /bin**	-l = Long listing, -a = All files, including hidden files
List all the files including hidden files in your current directory.	**ls -la**	
List all the files starting with letter ‘r’ in your current directory.	**ls r***	
List all the files having three characters in their names, from your current directory.	**ls ???**	? **wildcard** = Exactly one character
List all the files with extension .doc in your current directory.	**ls *.doc**	
List all the files having the first letter of their name within the range ‘l’ to ‘s’, from your current directory.	**ls [l-s]***	* = Any number of any characters
Create a file text1 and read its input from keyboard.	**echo "Type your input here:" > text1** **cat >> text1**	Press ENTER and Ctrl+D to terminate. The >> redirection operator directs the output of a command to the end of a file

Examples

Question	Command	Key points
Copy the contents of file text1 to another file text2.	**cp text1 text2**	
Append the contents of file text2 to file text1.	**cat text2 >> text1**	
Count the number of files in the current directory.	**ls -l | grep "^-" | wc -l**	**grep** "^-": Filters out only the lines that represent regular files (not directories or other types of files). In the long format, the lines representing regular files start with -. **wc** -l: Counts the number of lines.
Display the output of command ls –l to a file and on the output screen.	**ls -l | tee ls_output.txt**	**tee** command reads from standard input and writes to standard output and files simultaneously.
From file text1 print all lines starting from 10th line.	**tail -n +10 text1**	**tail** = Display EOF, -n: How many lines?, +10: Start from the 10th line (If we use 10, it will be the last 10 lines)

Examples

Question	Command	Key points
Find the number of users currently logged on to the system.	**who** **who | wc -l**	wc -l will give number of users, since it will count the lines of the output of the *who* command
Delete all the files with their names starting with “tmp”.	**rm tmp***	

Linux Editors

- vi – The original Linux editor
- vim – Improved version of vi
- emacs – Can be configured as per the programming language
- nano – Easy to work with (To install: **sudo apt-get update** and then **sudo apt-get install nano**)

- To install VS Code: **sudo snap install --classic code**
- To start: **code**

The vi Editor

- Modal editing: Three different modes: normal, insert, command-line
- **Normal mode**: Navigate the file, Editing commands are possible
- **Insert mode**: For inserting text
- **Command-line mode**: Editor commands

- Normal mode commands: Cursor movement using h (left), j (down), k (up), l (right), w (one word forward), b (one word back), gg (BOF), G (EOF)

Using vi

- Problem: Create a file test.txt, add three lines, save and exit and again reopen the file and edit the second line, save and exit.
- By default, vi starts in the normal mode, so press **i** to switch to insert mode and type three lines, then **ESC**, then type **:wq** and **ENTER** to save and exit
- Then reopen the file, use **j** twice to move cursor down two lines, press **i** to switch to insert mode, again repeat **ESC :wq ENTER**
- To quit vi without saving the changes, **ESC :q! ENTER**
- To display line numbers in vi, **ESC :set number ENTER**

Moving First Three Lines of a File to its End in vi

- Once inside vi, press the Esc key to ensure you are in command mode.
- Move the cursor to the beginning of the file by pressing gg.
- Enter visual mode by typing V (uppercase V).
- Move the cursor down to select the first three lines by pressing 3j or simply 3↓.
- Once the lines are selected, cut them by typing d.
- Move to the end of the file by typing G.
- Paste the cut lines by typing p.

Copy the 5th line and paste it above the first line in vi

- Once inside vi, press the Esc key to ensure you are in command mode.
- Move the cursor to the 5th line by typing 5G or simply 5gg.
- Enter visual mode by typing V (uppercase V).
- Move the cursor down to select the entire 5th line by pressing j or ↓.
- Once the line is selected, copy it by typing y.
- Move to the first line by typing gg.
- Paste the copied line above the first line by typing P (uppercase P).

Find and Replace in vi

- Forward search cdac: ESC and then /cdac (n to go to the next occurrence, N to go to the previous occurrence)
- Backward search cdac: ESC and then ?cdac (n to go to the previous occurrence, N to go to the next occurrence)

- Replace abc with xyz: ESC and then **:%s/abc/xyz/g**
- Note: g = global, if omitted, it will replace only the first occurrence

su and sudo Commands

- Linux is very strict about what users can do
- Ordinary users can work in their home directory and can perform operations related to their own work
- Many Linux operations require elevated (higher) privileges: Two commands can be used for this
- **sudo**: Superuser Do: Execute a command as the superuser – Own password is needed
  - Example: Update system package list: **sudo** apt update … apt = Advanced Package Tool
- **su**: Switch User: Switch the current user to another user (If no name is specified, switch to root) – Target user's password is required
  - Example: **su -** to switch to the root user and then **exit** to return to the previous user

Users

- Create a new user: **sudo adduser cdac**
- Assign a shell: **sudo usermod -s /bin/sh cdac**
- Delete a user: **sudo deluser cdac**
- Change a user’s password**: sudo passwd cdac**

/etc/passwd File

- This plaintext file maps user names to user ids
- Example:
- `root:x:0:0:root:/root:/bin/bash`
- `daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin`
- `bin:x:2:2:bin:/bin:/usr/sbin/nologin`
- `sys:x:3:3:sys:/dev:/usr/sbin/nologin`
- `cdac:x:65534:65534:cdac:/home/cdac:/bin/bash`
- One line per user having seven fields separated by colons

/etc/passwd File

- Field 1: User name
- Field 2: User’s encrypted password (Actual password is in the *shadow* file, if it contains x)
- Field 3: User ID (UID) – User’s representation in the kernel
- Field 4: Group ID (GID) – Determines file permissions
- Field 5: User’s real name
- Field 6: User’s home directory
- Field 7: User’s shell
- Note: The superuser (root) always has UID and GID of 0

File Ownership

- Every file created on Linux is owned by some user
- We can find the owner/user name when we run the ls -l command
- After the user name, we see the group name

```
-rw-rw-r-- 1 atul atul   10 Mar  8 16:40 course.txt
drwxrwxr-x 3 atul atul 4096 Mar 12 18:56 dbda
```

- Why is group name = user name?
- When we create a user, Linux automatically also creates a group by the same name and makes the user a part of that group
- But then, what is a group?

Group

- Group = Collection of users
- Helps manage access to files, directories, other system resources
- Example:
  - Alice and Bob are a part of DAC group
  - Carol and Dave are a part of AI group
  - Alice and Carol are a part of GuestLectures group
  - Carol and Dave are a part of CampusCommittee group
  - Alice, Bob, and Dave are a part of ExtraCurricular group
- Every user has a primary group and possibly secondary groups
- User’s primary group is mentioned in /etc/passwd file

Group

- Each group has a group ID (GID), internally maintained by Linux
- Find existing groups: **groups**
- Create/modify/remove groups: **groupadd**, **groupmod**, **groupdel** commands
  - **sudo groupadd marketing** Create a new group
  - **sudo groupmod -n sales marketing** Rename marketing to sales
  - **sudo groupdel sales** Remove the group
- Membership management
  - **sudo usermod -aG marketing john** Add a user john to marketing group
  - **sudo gpasswd -d marketing john** Remove user john from the group (*May not work due to permissions issue*)
- Find groups and their members: **cat /etc/group | sort**
- Find group names of a user: groups <username>, e.g. **groups dac**

File Attributes

Type	Owner permissions			Group permissions			Other users permissions		
- d l	r	w	x	r	w	x	r	w	x

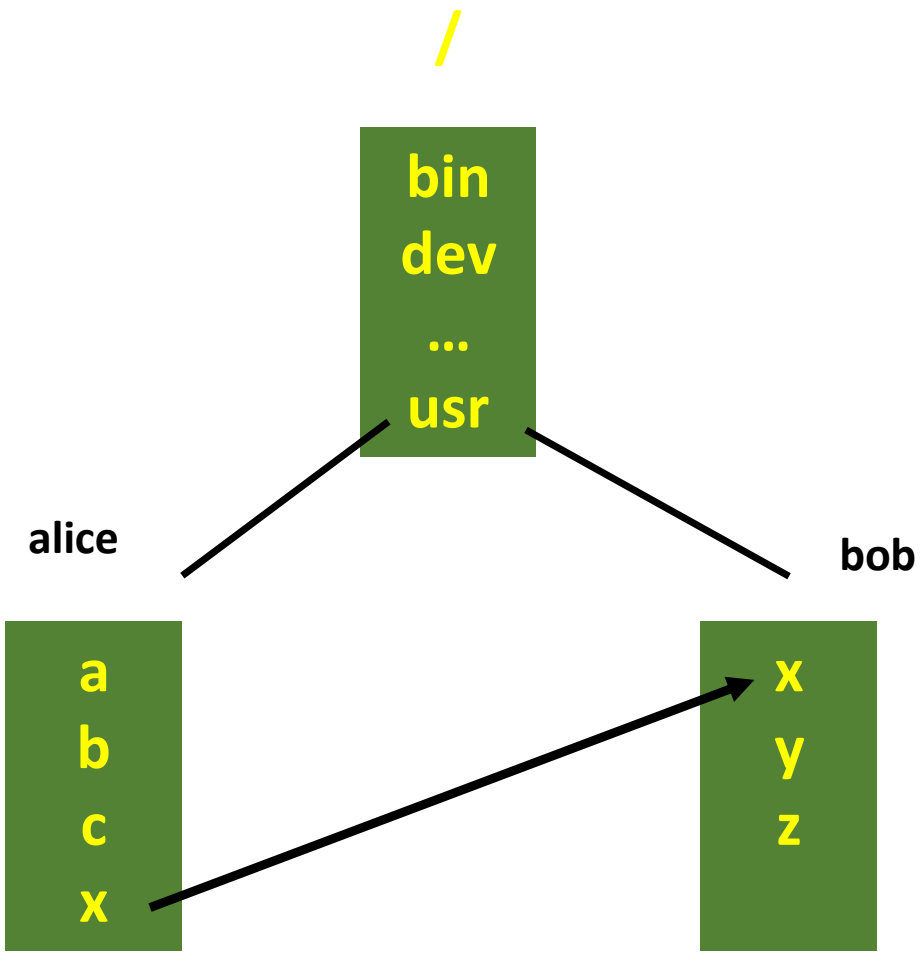
- Run ls -l command

```
drwxr-xr-x 2 atul atul 4096 Dec 11 17:35 Downloads
-rw-rw-r-- 1 atul both   12 Jan  6 14:32 hello.txt
```

- 10 characters at the beginning are called **file attributes**
- Character 1: d = directory, - = file, l = Symbolic link
- Next 3 characters: Owner permissions
- Next 3 characters: Group permissions
- Next 3 characters: Others permissions

File Links

- Problem: A user needs to refer to a file that belongs to another user
- Example: User Alice has a file, which user Bob needs to access
- User Alice will use a relative path, but user Bob will have to use an absolute path
- Solution: Use a **link**
- Types: **Hard link** and **Soft link**



Hard Links and Soft Links to Files

- **Hard link**
  - A direct pointer to actual data on the disk
  - Multiple hard links to the same file point to the same data
  - If the original file is deleted, the hard link continues to function because it still points to the same data on disk - The data is only deleted when all hard links (including the original) are removed
- **Soft link**
  - A pointer to the file name, not data (like a shortcut)
  - If the original file is deleted, the soft link becomes a "dangling" or "broken" link because it points to a file that no longer exists
- Hard link example: **ln cdac.txt cdac_hard_link**
- Soft Link example : **ln -s cdac.txt cdac_soft_link**

Creating a Hard Link

- Create a file using echo: **echo "This is some text." > FILE2**
- Create two more **hard links** to the file:
- **ln FILE2 FILE2_hardlink1**
- **ln FILE2 FILE2_hardlink2**
- Now, we have three hard links to the same file: *FILE2, FILE2_hardlink1, and FILE2_hardlink2*
- Check the inode number and link count for each of these names:
- **ls -i -l FILE2 FILE2_hardlink1 FILE2_hardlink2**

Creating a Symbolic link (Soft Link)

- Suppose we have a file named hello.txt
- To create a symbolic link: ln -s hello.txt symbolic_link
- Verify: ls -l
- Now you can say nano symbolic_link
- To remove the symbolic link: rm symbolic_link

File Permissions

For whom	
u	Owner/user
g	Owner group
o	Other users
a	All/Everyone

Operations	
+	Add permission
-	Remove permission
=	Reset everything else

Permissions	
r	Read
w	Write
x	Execute

chmod Command: Character-based

- chmod u+x file1: Add execute permission to user
- chmod g-w file1: Remove write permission from group
- chmod u+x,g+r test.txt: Add execute permission to user, read permission to group
- chmod a=r file1: Remove all existing permissions and only allow read to all

Octal File Permissions

Read	Write	Execute	Permission	Meaning	Binary	Decimal
0	0	0	- - -	No permission	**000**	**0**
0	0	1	- - x	Execute only	**001**	**1**
0	1	0	- w -	Write only	**010**	**2**
0	1	1	- w x	Write and Execute	**011**	**3**
1	0	0	r - -	Read only	**100**	**4**
1	0	1	r - x	Read and Execute	**101**	**5**
1	1	0	r w -	Read and Write	**110**	**6**
1	1	1	r w x	Read, Write, and Execute	**111**	**7**

chmod command: Octal

- chmod 700 file1: All the permissions only for the owner, no permissions for anyone else
- chmod 777 file1: All the permissions for everyone
- chmod 755 file1: rwx for user, rx and group, rx for others
- chmod 525 file1: rx for owner, w to group, rx for others
- chmod 621 file1: rw for owner, w to group, x for others

Linux Operating System

Course Assignment

Practical Lab: File System Commands & Shell Operations

Distribution Used: Ubuntu 22.04 LTS
Virtualization Platform: Oracle VirtualBox 7.0
Date: March 6, 2026

Section 1: Virtual Machine Installation

1.1 Prerequisites

Before installing Ubuntu, the following tools and resources were downloaded:

- Oracle VirtualBox 7.0 — Free, open-source hypervisor for Windows/macOS/Linux
- Ubuntu 22.04.3 LTS ISO — Downloaded from https://ubuntu.com/download/desktop
- Host system: Windows 11 with at least 8 GB RAM and 30 GB free disk space

1.2 VirtualBox Setup & VM Configuration

Step 1 — Install VirtualBox: Run the downloaded VirtualBox installer on Windows. Accept all default options and install. Restart if prompted.

Step 2 — Create a New Virtual Machine: Open VirtualBox and click 'New'. Configure as follows:

- Name: Ubuntu-22.04 | Type: Linux | Version: Ubuntu (64-bit)
- Memory (RAM): 4096 MB (4 GB)
- Virtual Hard Disk: Create new VDI → Dynamically allocated → 25 GB

Step 3 — Attach ISO: Go to VM Settings → Storage → Controller: IDE → Empty → Click disk icon → Choose the Ubuntu .iso file.

Step 4 — Adjust Display: Settings → Display → Set Video Memory to 128 MB. Enable 3D Acceleration.

1.3 Ubuntu Installation Steps

Step 5 — Boot the VM: Click 'Start'. The VM boots from the Ubuntu ISO and loads the GRUB/live menu.

Step 6 — Choose Install: Select 'Try or Install Ubuntu' from the boot menu.

Step 7 — Installation Wizard:

- Language: English → Click 'Install Ubuntu'
- Keyboard Layout: English (US)
- Updates: Select 'Normal Installation', check 'Download updates while installing'
- Installation Type: 'Erase disk and install Ubuntu' (safe inside VM)
- Time Zone: Set to your region (e.g., Asia/Kolkata)
- User Account: Enter name, username (e.g., student), and a strong password

Step 8 — Wait for installation (~10–15 minutes). Click 'Restart Now' when prompted. Remove the ISO if asked.

Step 9 — Post-install: Log in with credentials. Open a terminal with Ctrl+Alt+T. Run system update:

```
$ sudo apt update && sudo apt upgrade -y
```

Installation is now complete. The Ubuntu 22.04 LTS system is ready for use.

Section 2: Linux Command Demonstrations

Task 1: Display Current Username

Command Used:

```
$ whoami
```

Sample Output:

```
student
```

The whoami command prints the login name of the currently active user. This confirms the session is running as 'student'.

Task 2: Display Current Working Directory

Command Used:

```
$ pwd
```

Sample Output:

```
/home/student
```

pwd (Print Working Directory) outputs the absolute path of the current directory in the filesystem hierarchy.

Task 3: List All Files in Current Directory

Command Used:

```
$ ls
```

Sample Output:

```
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
```

The ls command lists all non-hidden files and directories in the current working directory.

Task 4: List Files with Permissions, Owner, and Size

Command Used:

```
$ ls -lh
```

Sample Output:

```
total 64K
drwxr-xr-x 2 student student 4.0K Mar  7 10:00 Desktop
drwxr-xr-x 2 student student 4.0K Mar  7 10:00 Documents
drwxr-xr-x 2 student student 4.0K Mar  7 10:00 Downloads
drwxr-xr-x 2 student student 4.0K Mar  7 10:00 Music
drwxr-xr-x 2 student student 4.0K Mar  7 10:00 Pictures
```

The -l flag enables long listing format showing permissions, link count, owner, group, size, modification date, and name. The -h flag makes sizes human-readable (K, M, G).

Task 5: Create a Directory Named 'practice'

Command Used:

```
$ mkdir practice
```

Verification:

```
$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos  practice
```

mkdir (Make Directory) creates a new directory. The new 'practice' folder is now visible in the listing.

Task 6: Navigate into the 'practice' Directory

Command Used:

```
$ cd practice
```

Verification:

```
$ pwd
/home/student/practice
```

The cd (Change Directory) command moves the shell session into the specified directory. pwd confirms the new location.

Task 7: Create Three Text Files

Command Used:

```
$ touch file1.txt file2.txt file3.txt
```

Verification:

```
$ ls
file1.txt  file2.txt  file3.txt
```

The touch command creates empty files if they do not exist. Multiple filenames can be provided in a single command.

Task 8: Display Contents of file1.txt Using cat

Commands Used:

```
$ echo "Hello from file1" > file1.txt
$ cat file1.txt
```

Sample Output:

```
Hello from file1
```

cat (Concatenate) reads and displays the contents of a file to standard output. First, content was written into the file using echo with output redirection (>).

Task 9: Write Text into a File Using echo

Command Used:

```
$ echo "Linux is a powerful open-source operating system." > notes.txt
$ cat notes.txt
```

Sample Output:

```
Linux is a powerful open-source operating system.
```

The echo command prints a string. The > operator redirects that output to a file (overwriting). Using >> would append instead.

Task 10: Copy file1.txt to backup.txt

Command Used:

```
$ cp file1.txt backup.txt
```

Verification:

```
$ ls
$ cat backup.txt
backup.txt  file1.txt  file2.txt  file3.txt  notes.txt
Hello from file1
```

cp (Copy) duplicates the source file to the specified destination. Both the original and the copy now exist simultaneously.

Task 11: Move backup.txt to a New Directory called 'backup'

Commands Used:

```
$ mkdir backup
$ mv backup.txt backup/
```

Verification:

```
$ ls backup/
backup.txt
```

First mkdir creates the target directory. Then mv (Move) moves the file into it. The original location no longer contains backup.txt.

Task 12: Rename file2.txt to data.txt

Command Used:

```
$ mv file2.txt data.txt
```

Verification:

```
$ ls
backup  data.txt  file1.txt  file3.txt  notes.txt
```

mv is used for both moving and renaming. When source and destination are in the same directory with different names, the file is effectively renamed.

Task 13: Remove a File Using rm

Command Used:

```
$ rm file3.txt
```

Verification:

```
$ ls
backup  data.txt  file1.txt  notes.txt
```

rm (Remove) permanently deletes a file. Unlike a GUI trash bin, this cannot be undone easily. Use rm -i for interactive confirmation before deletion.

Task 14: Remove an Empty Directory Using rmdir

Commands Used:

```
$ mkdir emptydir
$ rmdir emptydir
```

Verification:

```
$ ls
backup  data.txt  file1.txt  notes.txt
```

rmdir only removes directories that are completely empty. To remove a non-empty directory, use rm -r. This is a safety feature to prevent accidental data loss.

Task 15: Create a Directory Tree: project/data/logs

Command Used:

```
$ mkdir -p project/data/logs
```

Verification:

```
$ ls -R project/
project/:
data

project/data/:
logs

project/data/logs/:
```

The -p flag (parents) tells mkdir to create all intermediate directories that do not yet exist. Without -p, the command would fail if 'project' or 'project/data' did not already exist.

Task 16: Display the First 5 Lines of a File Using head

Setup & Command:

```
$ echo -e "Line 1\nLine 2\nLine 3\nLine 4\nLine 5\nLine 6\nLine 7" > sample.txt
$ head -n 5 sample.txt
```

Sample Output:

```
Line 1
Line 2
Line 3
Line 4
Line 5
```

head prints the first N lines of a file. The -n flag specifies the count. By default (without -n), head shows the first 10 lines.

Task 17: Display the Last 5 Lines of a File Using tail

Command Used:

```
$ tail -n 5 sample.txt
```

Sample Output:

```
Line 3
Line 4
Line 5
```

```
Line 6
Line 7
```

tail prints the last N lines of a file. It is commonly used to monitor log files in real time with the -f (follow) flag.

Task 18: Sort the Contents of a Text File

Setup & Command:

```
$ echo -e "banana\napple\ncherry\ndate" > fruits.txt
$ sort fruits.txt
```

Sample Output:

```
apple
banana
cherry
date
```

sort arranges lines alphabetically by default. Use sort -r for reverse order, sort -n for numerical sorting, or sort -u to remove duplicate lines.

Task 19: Search for the Word 'Linux' in a File Using grep

Setup & Command:

```
$ echo -e "Linux is great.\nI love Ubuntu.\nLinux commands are powerful." > info.txt
$ grep "Linux" info.txt
```

Sample Output:

```
Linux is great.
Linux commands are powerful.
```

grep (Global Regular Expression Print) scans the file line by line and prints lines that contain a match for the given pattern. It supports full regular expressions.

Task 20: Search Using grep with Case-Insensitive Option (-i)

Command Used:

```
$ grep -i "linux" info.txt
```

Sample Output:

```
Linux is great.
Linux commands are powerful.
```

The -i flag makes grep case-insensitive. Patterns like 'linux', 'LINUX', or 'LiNuX' would all match. This is useful when the case of the search term is unknown.

Task 21: Count Matching Lines Using grep -c

Command Used:

```
$ grep -c "Linux" info.txt
```

Sample Output:

```
2
```

The -c flag suppresses normal output and instead prints the count of matching lines. Here, 2 lines in info.txt contain the word 'Linux'.

Task 22: Display the Manual Page of the ls Command

Command Used:

```
$ man ls
```

Sample Output (partial):

```
LS(1)                    User Commands                    LS(1)

NAME
       ls - list directory contents

SYNOPSIS
       ls [OPTION]... [FILE]...

DESCRIPTION
       List information about the FILEs (the current directory by default).
       Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.

(Press q to quit the manual page)
```

man (Manual) displays the full reference manual for a command. It includes synopsis, description, all available options, and examples. man pages are the primary documentation resource in Linux.

Task 23: Find the Location of the grep Command Using which

Command Used:

```
$ which grep
```

Sample Output:

```
/usr/bin/grep
```

which searches the user's PATH environment variable and returns the full path of the executable for the given command. This confirms grep is installed at /usr/bin/grep.

Task 24: Locate a File Using the locate Command

Commands Used:

```
$ sudo updatedb
$ locate file1.txt
```

Sample Output:

```
/home/student/practice/file1.txt
```

locate searches a pre-built database of filenames for fast results. sudo updatedb refreshes this database first. Unlike find, locate is extremely fast but may not reflect very recent changes until updatedb is re-run.

Section 3: Command Summary Reference

Task #	Task Description	Command Used	Purpose / Notes
1	Display username	`whoami`	Shows current logged-in user
2	Current working directory	`pwd`	Prints absolute path of current directory
3	List directory contents	`ls`	Lists all non-hidden files and folders
4	List with details	`ls -lh`	Shows permissions, owner, size, date
5	Create directory	`mkdir practice`	Creates a new directory named practice
6	Navigate directory	`cd practice`	Changes current directory to practice
7	Create files	`touch file1.txt ...`	Creates empty files
8	Display file contents	`cat file1.txt`	Prints file contents to terminal

Task #	Task Description	Command Used	Purpose / Notes
9	Write to file	`echo "..." > file.txt`	Writes/overwrites text into a file
10	Copy file	`cp file1.txt backup.txt`	Duplicates a file to new name
11	Move file	`mv backup.txt backup/`	Moves file into another directory
12	Rename file	`mv file2.txt data.txt`	Renames file in same directory
13	Remove file	`rm file3.txt`	Permanently deletes a file
14	Remove empty dir	`rmdir emptydir`	Removes an empty directory only
15	Create dir tree	`mkdir -p project/data/logs`	-p creates all parent directories
16	First N lines	`head -n 5 sample.txt`	Displays first 5 lines of file
17	Last N lines	`tail -n 5 sample.txt`	Displays last 5 lines of file
18	Sort file contents	`sort fruits.txt`	Alphabetically sorts file lines
19	Search in file	`grep "Linux" info.txt`	Prints lines matching the pattern
20	Case-insensitive search	`grep -i "linux" info.txt`	Matches regardless of letter case
21	Count matches	`grep -c "Linux" info.txt`	Counts and prints number of matches
22	Manual pages	`man ls`	Displays full command documentation
23	Find command path	`which grep`	Shows full path of a command
24	Locate file	`locate file1.txt`	Fast file search using system database

End of Assignment

Shell Scripts

Variables

- User-defined variables
  - INSTITUTE=“ACTS”
  - echo “Hello $INSTITUTE”
- Shell variables/System Variables
  - echo "Home directory: $HOME"
  - echo "Username: $USER"
  - echo "Current directory: $PWD"
  - echo "Process ID: $$“

We need to use **** as escape character here

- Display the following text message on the monitor screen.
- Deposited $100 to you account: echo "Deposited \$100 to your account"

What is a Shell Script?

- **Shell script**: A file containing a series of commands
- Examples of commands:
  - echo “5/2” | bc
  - echo “scale=2; 5/2” | bc
  - echo “scale=10; 5/2” | bc
  - name=“Linux Student”
  - echo $name > out.txt

Writing a Shell Script – Pseudocode

- Open a new file with an extension .sh and write your pseudocode
- Example: Display the logged in user’s user id and also display if it is the root user or not

- nano get_user_name.sh

- # Get the user id

- # Check if user id is root or not

Writing a Shell Script – Shebang Operator

- **#!/bin/bash**

‘Shebang’ or ‘Shabang’ operator:
means sharp (as in C#)
! In comics is used as Bang!

- # Get the user id

- # Check if user id is root or not

Writing a Shell Script – Get the User Id

- #!/bin/bash

- # Get the user id
- **CURRENT_USER_NAME=$USER**
- **echo “Logged in user is: $CURRENT_USER_NAME”**

- # Check if user id is root or not

Executing a Shell Script

- Try running: get_user_name.sh
- Error!

- Specify the path using dot
- Try running: ./get_user_name.sh
- Error!

- Give execute permission to yourself
- chmod u+x get_user_name.sh
- Try running: ./get_user_name.sh

Complete the Shell Script, Save and Rerun

- #!/bin/bash

- # Get the user id
- CURRENT_USER_NAME=$USER
- echo “Logged in user is: $CURRENT_USER_NAME”

- # Check if user id is root or not
- **if ["$UID" -eq 0]; then**
- **echo "$**CURRENT_USER_NAME **is the root user."**
- **else**
- **echo "$**CURRENT_USER_NAME **is NOT the root user."**
- **fi**

Using Variables

- read_example_prompt.sh
- #!/bin/bash

- read -p "Enter your name: " name
- read -p "Enter your age: " age
- read -p "Enter your city: " city

- echo "My name is: $name"
- echo "My age is: $age"
- echo "My city is: $city"

Conditions – if Statement

- #!/bin/sh
- a=10
- b=20
- if [$a -eq $b]
- then
- echo “a is equal to b”
- fi
- if [$a -ne $b]
- then
- echo “a is not equal to b”
- fi

Same Example using if-else

- #!/bin/sh

- a=10
- b=20

- if [$a -eq $b]
- then
- echo “a is equal to b”
- else
- echo “a is not equal to b”
- fi

Check If a Number is Even or Odd

```
#!/bin/bash

read -p "Enter a number : " n

rem=$(( $n % 2 ))
# Other syntaxes
# let "rem = n % 2"
# rem=$(expr $n % 2)
# rem=$(echo "$n % 2" | bc)

if [ $rem -eq 0 ]
        then
                echo "$n is even number"
else
                echo "$n is odd number"
fi
```

if-elif-else

```
#!/bin/bash

read -p "Enter a Number: " num

if [ $num -lt 0 ]; then
    echo "Negative"
elif [ $num -gt 0 ]; then
    echo "Positive"
else
    echo "Neither Positive Nor Negative"
fi
```

Relational Operators (Only for Numeric Data)

- Equal to -eq
- Not Equal to -ne
- Less than -lt
- Greater than -gt
- Less than or Equal to -le
- Greater than or Equal to -ge

Max of three numbers

```
#!/bin/bash
echo "Enter Num1"
read num1
echo "Enter Num2"
read num2
echo "Enter Num3"
read num3
# To accept multiple inputs: read num1 num2 num3

if [ $num1 -gt $num2 ] && [ $num1 -gt $num3 ]
then
    echo $num1
elif [ $num2 -gt $num1 ] && [ $num2 -gt $num3 ]
then
    echo $num2
else
    echo $num3
fi
```

String Comparison

```
#!/bin/bash

# Shell script to check whether two
# input strings is equal

# take user input
echo "Enter string a : "
read a

echo "Enter string b : "
read b

# check equality of input
if [ "$a" == "$b" ]; then

    # if equal print equal
    echo "Strings are equal."
else
     # if not equal
    echo "Strings are not equal."
fi
```

Calculate Average using Bash Calculator

- #!/bin/bash

- read -p "Enter marks in subject 1: " marks1
- read -p "Enter marks in subject 2: " marks2
- read -p "Enter marks in subject 3: " marks3

- total=$(($marks1 + $marks2 + $marks3))
- # Another syntax: total=$(expr $marks1 + $marks2 + $marks3)
- # Another syntax: let total=$marks1+$marks2+$marks3
- # Another syntax: let total=marks1+marks2+marks3
- # Another syntax: total=$(echo "$marks1 + $marks2 + $marks3" | bc)

- echo "Total marks: $total"

- avg=$(echo "scale=4; $total / 3" | bc)

- echo "Average marks: $avg"

Input Must Have All Alphabets

- #!/bin/bash
- read -p "Enter a string: " input_string
- if [[$input_string =~ ^[[:alpha:]]+$]]; then
- echo "Input contains only alphabetic characters."
- else
- echo "Input contains non-alphabetic characters."
- fi

=~ is an operator in Bash to compare a string with a regular expression pattern

^ indicates start of the string
+ indicates one or more characters
$ indicates end of the string

Accept Only a 10-Digit Phone Number

```
#!/bin/bash

read -p "Enter a 10-digit number: " input

# Check if the input consists of exactly 10 digits using regex
if [[ $input =~ ^[0-9]{10}$ ]]; then
    echo "Valid input: $input"
else
    echo "Invalid input. Please enter a 10-digit number."
fi
```

Shell Program Containing Many File Operations

```
#!/bin/bash

# Create a directory if it doesn't exist
if [ ! -d "my_directory" ]; then
    mkdir my_directory
    echo "Directory 'my_directory' created."
fi

# Move into the directory
cd my_directory

# Create a new file
echo "Hello, world!" > my_file.txt
echo "Contents written to 'my_file.txt'."

# Display file content
echo "Contents of 'my_file.txt':"
cat my_file.txt

# Append to the file
echo "Appending more text." >> my_file.txt
echo "Additional content appended."

# Display file content again
echo "Contents of 'my_file.txt' after appending:"
```

C-Style for Loop

- #!/bin/bash
- # testing the C-style for loop – double brackets are needed
- for ((i=1; i <= 10; i++))
- do
- echo "The next number is $i"
- done

while Command

- #!/bin/bash
- # while command test
- var1=10
- while [$var1 -gt 0]
- do
- echo $var1
- var1=$[$var1 - 1]
- done

break Example

- #!/bin/bash
- # breaking out of a for loop
- for var1 in 1 2 3 4 5 6 7 8 9 10
- do
- if [$var1 -eq 5]
- then
- break
- fi
- echo "Iteration number: $var1"
- done
- echo "The for loop is completed"

PS Variables

- **Prompt Statement** Environment Variables
- PS1: Default interaction prompt
- PS2: Continuation interaction prompt
- PS3: Prompt used by the select statement inside a script

PS1: Default Interaction Prompt

- Try this command: export PS1="\u@\h \w> “
  - The default prompt will be changed to show the user name, home, and working directory
- export PS1="\u\w:\d-\t>"

- PS1="\$(date +%d.%m.%Y) > "
- export PS1

- PS1="\D{%F %T}>“

- export PS1="\u@\h [\$(date +%k:%M:%S)]> “

- To reset prompt, restart the terminal or **unset** ps1 or source **~/.bashrc**

PS2

- For incomplete commands, shell will display a secondary prompt as > and wait to complete the command and hit *Enter* again
- Example:
- echo “this is a
- > test”

- PS2=“continue …”
- echo “this is a
- continue... test”

The select Command and the PS3 (Prompt for Select command) Variable

- select_example_ps3.sh
- #!/bin/bash

- PS3="What is the day of the week?: "

- select day in mon tue wed thu fri sat sun;
- do
- echo "You selected $day"
- break
- done

```
atul@LAPTOP-E9C3P1LP:~/bash_course$ ./select_example_ps3.sh
1) mon
2) tue
3) wed
4) thu
5) fri
6) sat
7) sun
What is the day of the week?: 2
You selected tue
```

Problems and Solutions

- 1. List only the directories in your current directory
- ls -d */ | wc -l
- 2. Display the name and count of the directories in the current directory.
- ls -d */ | wc -l
- 3. Find out whether the users with a pattern“itp9” in their names have logged in ?
- who | grep 'itp9'
- 4. Find out whether a particular user “itp9” has logged in ?
- who | grep -w 'itp9'
- 5. Assign a value “Black” to var1 and then display the following message on the terminal using this variable: Black belt is associated with karate
- var1="Black"
- echo "${var1} belt is associated with karate"

Problems and Solutions

Create the file employee.txt having colon (:) separated fields.
The fields of the record are: enumber, ename, eposition, esal, edoj, edept.
And now answer the following:

a. List all the employees along with a row number

b. Sort the file as per the names

c. List top three salaried employees

d. Remove duplicate records from the file

e. List dept. no along with no. of employees working in each dept.

f. Sort the file in descending order of salary

- a. nl employee.txt
- b. sort -t':' -k2 employee.txt
- c. sort -t':' -k4nr employee.txt | head -n 3
- d. sort -u -t':' -k1,1 employee.txt
- e. while IFS=':' read -r _ _ _ _ _ edept; do
- echo "$edept"
- done < employee.txt | sort | uniq -c
- f. sort -t':' -k4nr employee.txt

Problems and Solutions

Accept a file name and a number (x). Display x lines from the top of the file.

Check if the file exists and is readable. The value of x should not exceed the total number of lines in the files. Display suitable messages in case an error is encountered.

Problems and Solutions

```
#!/bin/bash

# Function to display usage message
usage() {
    echo "Usage: $0 <file_name> <number_of_lines>"
}

# Check if correct number of arguments are provided
if [ "$#" -ne 2 ]; then
    usage
    exit 1
fi

# Assign command line arguments to variables
file_name="$1"
number_of_lines="$2"

# Check if file exists and is readable
if [ ! -r "$file_name" ]; then
    echo "Error: File '$file_name' does not exist or is not readable."
    exit 1
fi

# Check if number of lines is not greater than total lines in the file
total_lines=$(wc -l < "$file_name")
```

Problems and Solutions (LAB Exam)

Write a menu based script which displays the following options :

- Accept roll number, name, marks in sub1, sub2 and sub3. Calculate total, percentage, grade & class and enter the record in a file “student”. The marks are out of 100 in each subject. Allow the user to enter any number of records.

- Grade is found out as follows :
- Percentage Grade
- < 35 Fail
- >= 35 & < 50 Third
- >= 50 & < 60 Second
- >= 60 & < 75 First
- >= 75 Distn

- If the student secures < 35 in any one of the subjects, then class = “fail”. Otherwise
- class=”pass”.

Problems and Solutions

```
#!/bin/bash

# Function to calculate grade based on percentage
calculate_grade() {
  if (( $1 < 35 )); then
    grade="Fail"
  elif (( $1 >= 35 && $1 < 50 )); then
    grade="Third"
  elif (( $1 >= 50 && $1 < 60 )); then
    grade="Second"
  elif (( $1 >= 60 && $1 < 75 )); then
    grade="First"
  else
    grade="Distn"
  fi
}

# Function to calculate class
calculate_class() {
  if (( $1 < 35 || $2 < 35 || $3 < 35 )); then
    class="Fail"
  else
    class="Pass"
  fi
}
```

Problems and Solutions (NOT for Lab Exam, Just for Practice)

- Accept roll number from the user at the command line. Search it in the “student” file. If it is present, then display name, percentage, grade and class of the student. If the roll number is not present, then display a message “not found”. Allow the user to enter any number of queries.

Problems and Solutions

```
#!/bin/bash

# Function to search for roll number in student file
search_roll_number() {
  roll_number="$1"
  found=false
  while read -r line; do
    rno=$(echo "$line" | cut -d ' ' -f 1)
    if [[ "$rno" == "$roll_number" ]]; then
      found=true
      name=$(echo "$line" | cut -d ' ' -f 2)
      percentage=$(echo "$line" | cut -d ' ' -f 7)
      grade=$(echo "$line" | cut -d ' ' -f 8)
      class=$(echo "$line" | cut -d ' ' -f 9)
      echo "Roll Number: $roll_number"
      echo "Name: $name"
      echo "Percentage: $percentage"
      echo "Grade: $grade"
      echo "Class: $class"
      break
    fi
  done < student
  if [ "$found" == "false" ]; then
    echo "Roll number $roll_number not found."
  fi
```

Past Lab Exam Question

- Write a bash script to take input string from user. Print output as follows.
- a) If all the characters in input string is upper case, print "Input String is in UPPER case"
- b) If all the characters in input string is lower case, print "Input String is in LOWER case"
- c) else print "Input String is in MIXED case"

Past Lab Exam Question

```
#!/bin/bash

echo "Please enter a string: "
read input

# Check if the input is all uppercase
if [[ $input == [A-Z]* ]]; then
  echo "The input is in all uppercase."
# Check if the input is all lowercase
elif [[ $input == [a-z]* ]]; then
  echo "The input is in all lowercase."
# If it's not all uppercase or all lowercase, it must be mixed case
else
  echo "The input is in mixed case."
fi
```

Note: Some shells allow the = operator to be used for string comparison instead of ==, but it is not recommended.

We should use = for assignment and == for comparison.

Example:

```
my_variable="Hello"
```

and

```
if [ "$string1" == "$string2" ]; then
   echo "Strings are equal.“
fi
```

Past Lab Exam Question

- Write a bash script to take input Number as upper limit from user.
- a) Print the PID of the current script
- b) Print the Numbers in reverse till we reach zero
- e.g: If the user enters 5, print the numbers as below in separate lines
- 5
- 4
- 3
- 2
- 1
- c) Input Number range is greater than 1 and less than 100
- d) If user inputs number less than 1 or more than 100 print message “Input Number is Out of range”

Past Lab Exam Question

```
#!/bin/bash

echo "Enter the upper limit:"
read num

if [ $num -lt 1 ] || [ $num -gt 100 ]
then
    echo "Input Number is Out of range"
else
    echo "PID of the current script: $$"
    for (( i=$num; i>=1; i-- ))
    do
        echo $i
    done
fi
```

$$ is a special shell variable that represents the PID of the current process (i.e. running script)

Past Lab Exam Question

- Write a bash script to take two input string from user. Print the output as follows
- a) If the input string 1 and input String 2 matches, Print “The TWO STRING matches”. The match should not be case sensitive. Means Input String 1 is “SPARK” and input 2 string is “Spark”, still it should print “The TWO STRING matches”.
- b) If the input string 1 and input String 2 does not match, Print “The TWO STRING DOES NOT match”

Past Lab Exam Question

```
#!/bin/bash

echo "Enter the first string:"
read str1

echo "Enter the second string:"
read str2

if [ "${str1,,}" == "${str2,,}" ]
then
    echo "The TWO STRINGS MATCH"
else
    echo "The TWO STRINGS DO NOT MATCH"
fi
```

"${str1,,}": converts the string to lowercase

==: checks if two strings are equal

"${input^^}“: converts the string to uppercase

Past Lab Exam Question: Another Solution for the Same Problem

```
#!/bin/bash

echo "Enter the first string:"
read string1

echo "Enter the second string:"
read string2

# Convert both strings to lowercase and compare
lowercase_string1=$(echo "$string1" | tr '[:upper:]' '[:lower:]')
lowercase_string2=$(echo "$string2" | tr '[:upper:]' '[:lower:]')

if [ "$lowercase_string1" == "$lowercase_string2" ]; then
  echo "The strings are equal (case-insensitive)."
else
  echo "The strings are not equal (case-insensitive)."
fi
```

Shell Meta Characters

- **Shell meta characters**: Special characters that shell interprets to perform various operations rather than being treated as literal text
- Wildcard characters
  - * means 0 or more characters (e.g. **ls *.txt**)
  - ? means exactly one character (e.g. **ls file?.txt**)
- Redirection characters
  - > redirects to a file, overwriting the contents
  - >> redirects to a file, appending the contents
  - < takes input from a file (e.g. cat < file.txt)
  - | sends output of one command to the next (e.g. **ls | grep "file"**)
- Logical operators: && and ||
- Escaping: Ignores the meaning of the special character, e.g. **echo "Hello*"** will print Hello*
- Brace expansion: **echo {1..5}** will output 1 2 3 4 5

Shell Meta Characters

- Meta characters: Characters that have a special meaning
- Wildcards: * ? []
- Redirection: > >> < |
- Command substitution: $()
- Quoting: ‘ “
- Escape character: \
- Logical operators: && ||
- Background execution: &

Job Control

- Ability to manage multiple processes running in a shell session
  - Start processes in the foreground/background
  - Pause/suspend and resume processes
  - Manage multiple processes concurrently

Example

- atul@lubuntu:~$ **sleep 10**
- atul@lubuntu:~$ **sleep 60 &**
- [1] 1150
- atul@lubuntu:~$ **echo "hello there"**
- hello there
- atul@lubuntu:~$ **jobs**
- [1]+ Running sleep 60 &
- atul@lubuntu:~$ **fg**
- sleep 60

- *What are we doing?*
- Start a process in the foreground
- Then start a process in the background
- Then run another process in the foreground
- Check that the background process is still running
- Finally, bring the background process to the foreground

History

- List of previously executed shell commands
- Commands:
- **history**: Display a list of previously executed commands
- **!!**: Execute the last command again
- **!n**: Execute the command with index *n* from history

The **chown** command

- Change ownership of files/directories
- chown user:group filename – Change owner and group
- chown user filename – Change only owner
- Examples:
  - chown user1 example.txt
  - chown user1:group1 example.txt

Assignment

- Create a file cdac.txt
- Change the default permission of the file cdac.txt:
  - owner/user should have read, write, and execute permissions
  - group should only read and execute permissions
  - others should have no permissions

File Access Control List (ACL)

- **Access Control List (ACL)**: More fine-grained permissions that rwx
- Grant/deny permissions to specific users/groups
- View the ACL of a file: *getfacl file1*
- Set ACL: *setfacl -m u:user1:rwx file1* … Give rwx to user1 for file1
- Set ACL: *setfacl -m g:group1:rwx file1* … Give rwx to group1 for file1
- Remove ACL: *setfacl -x u:user1 file1*

Assignment

- Problem: Login as root, create a file 'filewithacl' and apply acl on it in such a way that only user5 is able to read and write in to it. Note: root will do all its work under 'dbda' folder.
- Solution:
- **su root**
- **cd /path/to/dbda**
- **touch filewithacl**
- **setfacl -x u:: filewithacl** # Remove any existing ACLs for other users
- **setfacl -m u:user5:rw filewithacl** # Grant read and write permissions to user5
- **setfacl -m o::--- filewithacl** # Deny all other users read and write permissions

The mount and unmount Commands

- **mount**: Attach a file system stored on a device such as HDD to a directory (also called mount point), so that the user can access it
- Example: sudo mount /dev/sdb1 /mnt/external
- This command mounts the filesystem from /dev/sdb1 to the directory /mnt/external
- sudo **unmount** /mnt/external … To remove the mounted filesystem

The Global Regular Expression Print (grep) Command

- Search for specific patterns (**regular expressions**) in text
- **grep CDAC hello.txt** Search for the word CDAC in hello.txt file
- **ls | grep ct** List files whose name contains ct anywhere
- **ls | grep ^Pi** List files whose name starts with Pi
- **ls | grep es$** List files whose name ends with es

Common grep Options

Option	Purpose	Example
-i	Ignore case sensitivity	grep -i "linux" file.txt
-v	Invert match (Show lines that to not match)	grep -v "error" file.txt
-c	Count number of matching lines	grep -c "success" file.txt
-n	Show line numbers for matches	grep -n "warning" file.txt
-l	Lowercase L … Show only file names with matches	grep -l "failed" *.log
-w	Match whole words only	grep -w "hello" file.txt
-r	Recursive search in directories	grep -r "TODO" /path/to/dir

Examples

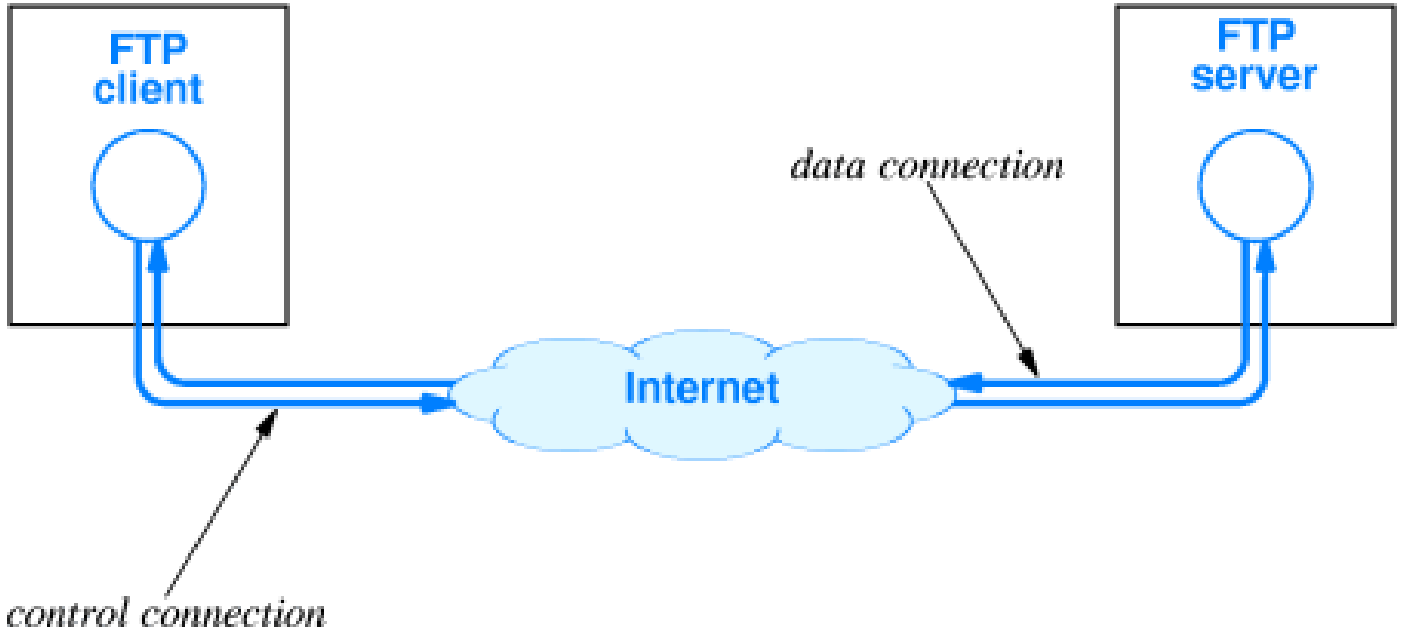
- grep -i "gatsby" greatgatsby.txt
- grep -v "gatsby" greatgatsby.txt
- grep -c "party" greatgatsby.txt
- grep -n "daisy" greatgatsby.txt
- grep -l "Nick" greatgatsby.txt
- grep -w "party" greatgatsby.txt
- grep -r "gatsby" novels/*.txt

find Command

- Search for files and directories
- Find a file based on its name: **find -name one.txt**
- Find all files with extension .txt: **find -type f -name "*.txt“**
- Find all files with size > 10kb: **find -type f -size +10k**
- Same, but also see the actual size > 10kb: **find -type f -size +10k –ls**
- Find directories with permission 755: **find -type d -perm 755 -ls**

FTP, Telnet, etc

FTP Concept


FTP
client
FTP
server
data connection
Internet
control connection

FTP (File Transfer Protocol)

- One of the earliest Internet protocols for file transfers
- Allows downloading (server-to-client) or uploading (client-to-server) of files remotely
- Made up of two parts
  - FTP server: Has files that can be downloaded, or it can also receive files from the users
  - FTP client: Downloads the require files from an FTP server, or it can upload files to the FTP server also

Setting up FTP Server on Ubuntu (on VirtualBox)

- Step 1: A common open-source FTP utility used in Ubuntu is vsftpd.
- **sudo apt install vsftpd**

- Step 2: Start the FTP server
- **sudo systemctl start vsftpd**

- Step 3: Check status
- **sudo systemctl status vsftpd**

- Step 4: Backup configuration files for safety
- **sudo cp /etc/vsftpd.conf /etc/vsftpd.conf_default**

Setting up FTP Server on Ubuntu

- Step 4: Create an FTP user, so that a client can access the FTP server
- **sudo useradd -m testuser**
- **sudo passwd testuser**

- Step 5: Allow FTP through the firewall
- **sudo ufw allow 20/tcp**
- **sudo ufw allow 21/tcp**

- Step 6: Connect to the FTP server locally
- **sudo ftp Ubuntu**
- (Here, Ubuntu is considered to be our Linux machine name, so replace it with the actual machine name … Find it using the command **echo $HOSTNAME**)

Accessing the FTP Server from our Windows Machine

- Open command prompt and type **ftp**
- ftp> **open**
- To 192.168.0.24 … This is the IP address of the Ubuntu machine – to find it, we can type *ip a* or *ifconfig* on the Ubuntu machine's terminal
- Connected to 192.168.0.24.
- 220 (vsFTPd 3.0.5)
- 200 Always in UTF8 mode.
- User (192.168.0.24:(none)): testuser
- 331 Please specify the password.
- Password:
- 230 Login successful.

Accessing the FTP Server from our Windows Machine

- ftp> stat
- Connected to 192.168.0.24.
- Type: ascii; Verbose: On ; Bell: Off ; Prompting: On ; Globbing: On
- Debugging: Off ; Hash mark printing: Off .
- ftp> pwd
- 257 "/home/testuser" is the current directory
- ftp> dir

Checking for Files

- Create a file on the FTP server (i.e. Ubuntu machine)
- atul@atul-virtualbox:~$ **sudo su**
- root@atul-virtualbox:/home/atul# **cd /home/testuser/**
- root@atul-virtualbox:/home/testuser# **nano test.txt**

Now Check in Windows FTP Client

- ftp> pwd
- 257 "/home/testuser" is the current directory
- ftp> dir
- 200 PORT command successful. Consider using PASV.
- 150 Here comes the directory listing.
- -rw-r--r-- 1 0 0 14 Mar 23 19:10 test.txt
- 226 Directory send OK.
- ftp: 69 bytes received in 0.01Seconds 8.63Kbytes/sec.

Download a File using FTP

- ftp> get test.txt
- 200 PORT command successful. Consider using PASV.
- 150 Opening BINARY mode data connection for test.txt (14 bytes).
- 226 Transfer complete.
- ftp: 14 bytes received in 0.00Seconds 14000.00Kbytes/sec.

Telnet and SSH

- **Telnet (Terminal + Network)** is an Internet protocol for allowing a client computer to connect to a remote server computer and work on the server computer
- Somewhat obsolete (outdated)
- Now we generally use **SSH (Secure Shell)** instead of Telnet
- SSH also allows a remote client computer to connect to a server over the Internet
  - Very useful in **cloud computing**

Setting up Telnet Server on Ubuntu

- Step 1: Install Telnet server
- sudo apt install telnetd -y

- Step 2: Check the Telnet status
- sudo systemctl status inetd

- Step 3: Configure firewall for Telnet
- sudo ufw enable
- sudo ufw allow 23

- Step 4: Test
- telnet 192.168.0.24

Accessing Telnet from Windows

- On the command prompt, type telnet
- If you get an error, search for *Turn Windows Features On/Off* and enable *Telnet* there, then retry telnet
- On the telnet prompt, o 192.168.0.24
- User id: testuser, Password: testuser

Setting up SSH on Ubuntu Server

- Step 1: Install SSH server
- **sudo apt install openssh-server**

- Step 2: Check the SSH status
- **sudo systemctl status ssh**

- Step 3: Configure firewall for SSH
- **sudo ufw allow ssh**

- Step 4: Restart if necessary
- **sudo service ssh restart**

Access SSH using Putty

- Download and install **Putty** (https://www.putty.org/)
- Open Putty, enter IP address as IP address of the Ubuntu server (e.g. 192.168.0.24) and connect
- User id: testuser, Password: testuser